Journal of Computational and Applied Mathematics 274 (2015) 44–57

Contents lists available at ScienceDirect

Journal of Computational and Applied Mathematics

journal homepage: www.elsevier.com/locate/cam

An alternative full-pivoting algorithm for the factorization of indefinite symmetric matrices

I. Fernández de Bustos *, J. Agirrebeitia, G. Ajuria, R. Ansola

Department of Mechanical Engineering, Escuela Técnica Superior de Ingeniería de Bilbao, Spain

ARTICLE INFO

Article history:
Received 26 February 2013
Received in revised form 26 February 2014

MSC:
65F05
15A06
15A12

Keywords:
Symmetric indefinite matrices
Matrix factorization
Bunch–Kaufman
Bunch–Parlett

ABSTRACT

This paper presents an algorithm for the factorization of indefinite symmetric matrices that factors any symmetric matrix A into the form LDL^T, with D diagonal and L triangular, with its subdiagonal filled with zeros. The algorithm is based on Jacobi rotations, as opposed to the widely used permutation methods (Aasen, Bunch–Parlett, and Bunch–Kaufman). The method introduces little increase in computational cost and provides a bound on the elements of the reduced matrices of order $2\sqrt{n}f(n)$, which is smaller than that of the Bunch–Parlett method ($\simeq 3nf(n)$), and similar to that of Gaussian elimination with full pivoting ($\sqrt{n}f(n)$). Furthermore, the factorization method is not blocked. Although the method presented is formulated in a full-pivoting scheme, it can easily be adapted to a scheme similar to that of the Bunch–Kaufman approach. A backward error analysis is also presented, showing that the elements of the error matrix can be bounded in terms of the elements of the reduced matrices.

© 2014 Elsevier B.V. All rights reserved.

1. Introduction

The most widely accepted algorithm for the direct solution of symmetric indefinite systems of linear equations is the Bunch–Kaufman factorization [1,2]. Even though it has received some criticism in recent investigations [1], and it has been demonstrated that its bound is tight [3], it is still the algorithm of choice for most applications, mainly because it solves most of the usual situations with great accuracy. Recently, an observation by Demmel, Higham, and Schreiber [4–6], raised a question on the stability of the method, which was related to its block nature [1], but this question was successfully solved by Higham [2]. In this last reference, the author demonstrates that, for any kind of pivoting, the algorithm is stable if the elements on the triangularized matrices are bounded, but also states that the accuracy of the method is determined by the actual bound on the elements that appear in the factorization. This means that partial pivoting methods, although faster and stable, present lower accuracy.

The Bunch–Kaufman algorithm [7] can be considered as a partial-pivoting approach to the Bunch–Parlett algorithm [8], which reduces the order of the cost of the comparisons performed in pivoting from $O(n^3)$ to $O(n^2)$ by performing comparisons in only two rows, and thus performing partial pivoting instead of full pivoting. This also leads to a decrease in precision, and thus, sometimes is preferable to use a full pivoting scheme. Furthermore, the relative operative cost of floating point operations when compared to memory accesses has dramatically fallen in the last years, which could lead to the situation in a near future where the location of the optimal pivot could be performed along with the Schur factorization with little or no penalty at all, thus reaching a situation where full pivoting could arrive at no cost in computational time.

* Corresponding author. Tel.: +34 946014093; fax: +34 946014215.
E-mail address: igor.fernandezdebustos@ehu.es (I. Fernández de Bustos).

http://dx.doi.org/10.1016/j.cam.2014.07.003
0377-0427/© 2014 Elsevier B.V. All rights reserved.

The key idea of all of these methods is to preserve symmetry in the system, in order to keep both the computational and the storage cost as low as possible. This can be achieved not only by diagonal pivoting but also by Jacobi rotations. The use of Jacobi rotations, although widely used for other factorizations, has not been widely considered before for matrix triangularization [9]. The great advantage of this approach is that one always reaches a pivot that is greater than or equal to the largest element of the submatrix to be factored, and, thus, the bound on the growth of the elements that appear in the factorization process is lower. But the use of rotations in the process is not taken into account in the work presented by Higham [2], and thus, a backward error analysis is required to demonstrate stability. This paper presents an algorithm for the triangularization of symmetric matrices using Jacobi rotations instead of diagonal pivoting, along with its backward error analysis.

2. Organization of the paper

First, the algorithm is presented, and then the stability of the method is proven by means of a backward error analysis. A bound on the maximum absolute values of the elements that appear in the process is demonstrated. It is shown that these elements are bounded by an expression of order $2\sqrt{n}f(n)$. An operation count is also performed. This covers the factorization itself. The solution of linear systems of equations using the algorithm presented here is then addressed, by showing that the overall procedure (triangularization and backward solution) is stable and by performing an operation count of the procedure for solution of the linear system. Finally, some numerical results are presented, and some conclusions drawn. To make a fair comparison, we take only full-pivoting algorithms into account, leaving partial-pivoting schemes for future work, although the results obtained with the Bunch–Kaufman algorithm are also included. The algorithms are based on Schur reduction, and thus the paper focuses on the pivot-reduction scheme for a given submatrix. For nomenclature, $\mathbf{A}_r$ is used to denote the submatrix of dimension r, and $\mathbf{B}_r$ to denote the permutated, rotated, and reduced matrix (so that $\mathbf{A}_{r-1}$ is the submatrix of $\mathbf{B}_r$ with the first row and column eliminated). $t_r = \tan(\theta)$ is the rotation that exactly zeroes the element b_{01}, and $\bar{t}_r$ is the rotation obtained in finite precision.

3. Algorithm description

The aim is to factor a given matrix A in the form:

$$\mathbf{A} = \mathbf{LDL}^T = \begin{bmatrix} 1 & 0 & 0 & \cdot \\ 0 & 1 & 0 & \cdot \\ l_{20} & 0 & 1 & \cdot \\ l_{30} & l_{31} & 0 & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} d_{00} & 0 & 0 & \cdot \\ 0 & d_{11} & 0 & \cdot \\ 0 & 0 & d_{22} & \cdot \\ 0 & 0 & 0 & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} 1 & 0 & l_{20} & l_{30} & \cdot \\ 0 & 1 & 0 & l_{31} & \cdot \\ 0 & 0 & 1 & 0 & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{bmatrix}. \tag{1}$$

Obviously, the aforementioned factorization is only possible in the definite case. To perform a similar factorization for any given matrix, we use rotated pivoting, which is composed of a permutation of a couple of rows/columns and afterwards a rotation of them. Usually, when permutation is used, the equation to describe the factorization is in the form:

$$\mathbf{A} = \mathbf{PLDL}^T\mathbf{P}^T. \tag{2}$$

So, in the case of combinations of rotations and permutations, we will use the general equation:

$$\mathbf{A} = \mathbf{GLDL}^T\mathbf{G}^T \tag{3}$$

where $\mathbf{G}$ represent a permutation/rotation matrix. We must recall that, in this case, the first subdiagonal of $\mathbf{L}$ is composed of zeros.

The factorization is performed by means of Schur updates. For a given step in the permutation, we have a given submatrix, A_r, and we want to perform the update:

$$\mathbf{A}_r = \mathbf{G}_r\mathbf{P}_r\mathbf{L}_r\bar{\mathbf{A}}_{r-1}\mathbf{L}_r^T\mathbf{P}_r^T\mathbf{G}_r^T. \tag{4}$$

Being:

$$\mathbf{G}_r = \begin{bmatrix} \cos(\theta) & \sin(\theta) & \mathbf{0}_{r-2}^T \\ -\sin(\theta) & \cos(\theta) & \mathbf{0}_{r-2}^T \\ \mathbf{0}_{r-2} & \mathbf{0}_{r-2} & \mathbf{I}_{r-2} \end{bmatrix} \tag{5}$$

$$\mathbf{L}_r = \begin{bmatrix} 1 & 0 & \mathbf{0}_{r-2}^T \\ 0 & 1 & \mathbf{0}_{r-2}^T \\ \boldsymbol{l}_r & \mathbf{0}_{r-2} & \mathbf{I}_{r-2} \end{bmatrix} \tag{6}$$

$$\bar{\mathbf{A}}_{r-1} = \begin{bmatrix} d_{n-r,n-r} & \mathbf{0}_{r-1}^T \\ \mathbf{0}_{r-1} & \mathbf{A}_{r-1} \end{bmatrix} \tag{7}$$

$$\boldsymbol{l}_r = \begin{Bmatrix} l_{n-r+2,n-r} \\ l_{n-r+3,n-r} \\ l_{n-r+4,n-r} \\ \cdots \end{Bmatrix} \tag{8}$$

and $\mathbf{P}_r$ the pivoting matrix for the current iteration. The repeated application of Eq. (4) from $r = n$ to $r = 2$ delivers the full factorization. In order to store the pivot indexes, a $2n$ sized integer vector is arranged. The rotations can be safely stored in the zeros that appear in the subdiagonal of the factorization. This implies a minimal increase in memory consumption when compared to the Bunch–Parlett algorithm, which requires a n vector to store the pivots.

In order to explain the procedure for performing the update equation, Eq. (4), the indexing in $\mathbf{A}_r$ is relative to the currently processed submatrix, i.e., the indexes range from 0 to $r - 1$. Row major order, in which $j \geq i$, is used. aij is used for the elements of the original matrix and b_{ij} for the elements of the updated matrix.

Let a_{ij} be the element of $\mathbf{A}_r$ with maximum absolute value. If $j = i$, then piv[0] $= i$ and piv[1] $= 1$. If $j > i$, we rotate the elements in such a way that $t \leq 1$ and $|b_{ii}| > \left|b_{jj}\right|$. Thus, if $|a_{ii}| > \left|a_{jj}\right|$, then piv[0] $= i$ and piv[1] $= j$; otherwise, if $|a_{ii}| < \left|a_{jj}\right|$, piv[0] $= j$ and piv[1] $= i$, so that $a_{01} = 0$. We do this as suggested by [10]:

$$\beta = \frac{a_{ii} - a_{jj}}{a_{ij}}, \tag{9}$$

$$\tan(\theta) = t = \frac{\operatorname{sgn}(\beta)}{\beta + \sqrt{\beta^2 + 1}}, \tag{10}$$

$$s = \frac{t}{\sqrt{t^2 + 1}}, \tag{11}$$

$$\rho = \frac{t}{\sqrt{t^2 + 1} + 1}. \tag{12}$$

One has to take into account the fact that, if $a_{ii} = a_{jj}$, in this case the rotation in absolute value is such that $t = 1$. As the biggest absolute value is required for the element in b_{00}, one must use $t = 1$ if $a_{ii} = a_{jj} < 0$ and $t = -1$ if $a_{ii} = a_{jj} > 0$. It is also advisable to consider the situation of $a_{ij} = 0$, where no rotation should be performed.

The update equations are, thus:

$$b_{00} = a_{00} - ta_{01}, \tag{13}$$
$$b_{11} = a_{11} + ta_{01}, \tag{14}$$
$$b_{01} = 0, \tag{15}$$
$$b_{0k} = a_{0k} - s(a_{1k} + \rho a_{0k}), \tag{16}$$
$$b_{1k} = a_{1k} + s(a_{0k} - \rho a_{1k}). \tag{17}$$

As exposed before, in order to reduce storage, we take advantage of the fact that b_{01} is equal to zero, and we store t in b_{01} instead of a zero.

After this, the only remaining operation to be done in the iteration is to perform a Schur reduction, taking into account the fact that $b_{01} = 0$ and, thus, row 1 has already been processed:

$$b_{kl} = a_{kl} - \frac{b_{0k} b_{0l}}{b_{00}}, \tag{18}$$

$$b_{0k} = \frac{b_{0k}}{b_{00}}. \tag{19}$$

Obviously, we are using b_{ij} to name the elements of the matrix after the iteration, but they are all stored in the original matrix. The process is repeated until $r = 1$ or $|\mathbf{A}_r| = 0$.

As a summary, the algorithm can be described as Algorithm 1, where ROTATE(m,r,i,j) can be written as exposed in Algorithm 2

4. Backward error analysis of the factorization

The backward error analysis requires the determination of bounds on an error matrix $\mathbf{F}$, such as

$$\mathbf{A} + \mathbf{F} = \mathbf{M}\mathbf{D}\mathbf{M}^T, \tag{20}$$

where

$$\mathbf{M} = \prod_{r=2}^{n} \mathbf{P}_r \mathbf{M}_r. \tag{21}$$

Here, $\mathbf{P}_r$ is a permutation matrix and

$$\mathbf{M}_r = \begin{bmatrix} \mathbf{I}_{(n-r)x(n-r)} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{G}_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix} \begin{bmatrix} \mathbf{I}_{(n-r)x(n-r)} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & 1 & 0 & \mathbf{1} \\ \mathbf{0} & 0 & 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{m}_r & \mathbf{0} & \mathbf{I} \end{bmatrix}, \tag{22}$$

where $\mathbf{G}_r$ is a 2×2 rotation matrix.

Algorithm 1 Triangularization process

```
for r = n to 2 do
  m ← &A[n − r : n − r] {m points to the currently processed submatrix}
  p ← piv[2 ∗ (n − r)] {p points to the current indexes}
  let a_ij be the largest magnitude element in submatrix pointed by m
  if a_ij == 0 then
    p[0] ← 0 {nothing to do if the matrix is full of zeros, the factorization is terminated. One just need to store the pivots}
    p[1] ← 0
    return
  end if
  if i == j then
    p[0] ← i
    p[1] ← 1 {In the pivoting indexes, we use a relative numbering, i.e: we store the index of the row relative to the
    currently processed row/column}
  else
    if a_ii is larger than a_jj then
      p[0] ← i
      if j! = 0 then
        p[1] ← j
      else
        p[1] ← i {we do this to avoid the situation where a switch of the row i implies a change in j}
      end if
    else
      p[0] ← j
      if i! = 0 then
        p[1] ← i
      else
        p[1] ← j {same problem as before}
      end if
    end if
  end if
  ROTATE(m,r,p[0],p[1])
  SCHURREDUCE(m,r) {This reduction can easily be performed using dspr and dscal. It is important to take into account
  that m[1,2] is 0 and, thus, we can omit the reduction of row 1 of m}
end for
```

For a single step in the factorization, we can write the following for the submatrix to be reduced, the size of which is $r \times r$:

$$\mathbf{A}_r = \begin{bmatrix} a_{00r} & a_{01r} & \mathbf{a}_{0r}^t \\ a_{01r} & a_{11r} & \mathbf{a}_{1r}^t \\ \mathbf{a}_{0r} & \mathbf{a}_{1r} & \mathbf{A}_{rs} \end{bmatrix}. \tag{23}$$

The rotation leads to

$$\mathbf{G}_r^t \mathbf{A}_r \mathbf{G}_r = \begin{bmatrix} b_{00r} & 0 & \bar{\mathbf{a}}_{0r}^t \\ 0 & b_{11r} & \bar{\mathbf{a}}_{1r}^t \\ \bar{\mathbf{a}}_{0r} & \bar{\mathbf{a}}_{1r} & \mathbf{A}_{rs} \end{bmatrix}. \tag{24}$$

In finite precision, we obtain, instead,

$$\mathbf{G}_r^t \mathbf{A}_r \mathbf{G}_r = \begin{bmatrix} b_{00r} & 0 & \bar{\mathbf{a}}_{0r}^t \\ 0 & b_{11r} & \bar{\mathbf{a}}_{1r}^t \\ \bar{\mathbf{a}}_{0r} & \bar{\mathbf{a}}_{1r} & \mathbf{A}_{rs} \end{bmatrix} + \begin{bmatrix} \epsilon_{00r} & \epsilon_{01r} & \boldsymbol{\epsilon}_{0r1}^t \\ \epsilon_{01r} & \epsilon_{11r} & \boldsymbol{\epsilon}_{1r}^t \\ \boldsymbol{\epsilon}_{0r1} & \boldsymbol{\epsilon}_{1r} & \mathbf{0} \end{bmatrix}. \tag{25}$$

The next operation to be performed is

$$\mathbf{G}_r^t \mathbf{A}_r \mathbf{G}_r = \begin{bmatrix} 1 & 0 & \mathbf{0} \\ 0 & 1 & \mathbf{0} \\ \mathbf{m}_{0r} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix} \begin{bmatrix} b_{00r} & 0 & \mathbf{0} \\ 0 & b_{11r} & \bar{\mathbf{a}}_{1r}^t \\ \mathbf{0} & \bar{\mathbf{a}}_{1r} & \mathbf{B}_r \end{bmatrix} \begin{bmatrix} 1 & 0 & \mathbf{m}_{0r}^t \\ 0 & 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix}, \tag{26}$$

which is performed in two steps. The first step is

$$\mathbf{m}_r = \frac{1}{b_{00r}} \bar{\mathbf{a}}_{0r}, \tag{27}$$

Algorithm 2 Algorithm for function ROTATE

```
INTERCHANGERC(m,t,i,1){Simple interchange of rows/columns}
INTERCHANGERC(m,t,j,2)
aki ← m[1, 1]
akj ← m[2, 2]
aij ← m[1, 2]
b ← (0.5*(akj−aki))/aij
if b > 0 then
  tan ← 1./|b+√(b**2+1)|
else
  if aki < 0 then
    tan ← 1
  else
    tan ← −1
  end if
end if
ROTATEROWS12(m,t,tan) {a simple rotation that can be performed using BLAS calls}
m[1, 2] ← tan {We take advantage of the fact that m[0,1] is known to be one to store the magnitude of the rotation performed}
```

which, in finite precision, leads to

$$\mathbf{m}_r = \frac{1}{b_{00r}}\bar{\mathbf{a}}_{0r} + \boldsymbol{\epsilon}_{0r2}. \tag{28}$$

The second step is

$$\mathbf{B}_r = \mathbf{A}_{rs} - \mathbf{a}_{0r}^t \mathbf{m}_r, \tag{29}$$

which, in finite precision, is

$$\mathbf{B}_r = \mathbf{A}_{rs} - \mathbf{a}_{0r}\mathbf{m}_r^t + \boldsymbol{\epsilon}_{rs} = \mathbf{A}_{rs} - \mathbf{m}_r b_{00r}\mathbf{m}_r^t + \boldsymbol{\epsilon}_{0r2} b_{00r}\mathbf{m}_r^t + \boldsymbol{\epsilon}_{rs}. \tag{30}$$

So,

$$\mathbf{A}_r + \mathbf{F}_r = \mathbf{G}_r \begin{bmatrix} 1 & 0 & \mathbf{0} \\ 0 & 1 & \mathbf{0} \\ \mathbf{m}_{0r} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix} \begin{bmatrix} b_{00r} & 0 & \mathbf{0} \\ 0 & b_{11r} & \bar{\mathbf{a}}_{1r}^t \\ \mathbf{0} & \bar{\mathbf{a}}_{1r} & \mathbf{B}_r \end{bmatrix} \begin{bmatrix} 1 & 0 & \mathbf{m}_{0r}^t \\ 0 & 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix} \mathbf{G}_r^t, \tag{31}$$

where

$$\mathbf{F}_r = \mathbf{G}_r \begin{bmatrix} \epsilon_{00r} & \epsilon_{01r} & \boldsymbol{\epsilon}_{0r1}^t + \boldsymbol{\epsilon}_{0r2}^t \\ \epsilon_{01r} & \epsilon_{11r} & \boldsymbol{\epsilon}_{1r}^t \\ \boldsymbol{\epsilon}_{0r1} + \boldsymbol{\epsilon}_{0r1} & \boldsymbol{\epsilon}_{1r} & \boldsymbol{\epsilon}_{rs} + \boldsymbol{\epsilon}_{0r2} b_{00r}\mathbf{m}_r^t \end{bmatrix} \mathbf{G}_r^t. \tag{32}$$

If the two steps are concatenated,

$$\mathbf{A}_{r-1} + \mathbf{F}_{r-1} = \mathbf{G}_{r-1} \begin{bmatrix} 1 & 0 & \mathbf{0} \\ 0 & 1 & \mathbf{0} \\ \mathbf{m}_{0(r-1)} & \mathbf{0} & \mathbf{I}_{(r-3)x(r-3)} \end{bmatrix} \begin{bmatrix} b_{00(r-1)} & 0 & \mathbf{0} \\ 0 & b_{11(r-1)} & \bar{\mathbf{a}}_{1(r-1)}^t \\ \mathbf{0} & \bar{\mathbf{a}}_{1(r-1)} & \mathbf{B}_{r-1} \end{bmatrix} \begin{bmatrix} 1 & 0 & \mathbf{m}_{0(r-1)}^t \\ 0 & 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{(r-3)x(r-3)} \end{bmatrix} \mathbf{G}_{r-1}^t, \tag{33}$$

where

$$\mathbf{F}_{r-1} = \mathbf{G}_{r-1} \begin{bmatrix} \epsilon_{00(r-1)} & \epsilon_{01(r-1)} & \boldsymbol{\epsilon}_{0(r-1)1}^t + \boldsymbol{\epsilon}_{0(r-1)2}^t \\ \epsilon_{01(r-1)} & \epsilon_{11(r-1)} & \boldsymbol{\epsilon}_{1(r-1)}^t \\ \boldsymbol{\epsilon}_{0(r-1)1} + \boldsymbol{\epsilon}_{0(r-1)1} & \boldsymbol{\epsilon}_{1(r-1)} & \boldsymbol{\epsilon}_{(r-1)s} + \boldsymbol{\epsilon}_{0(r-1)2} b_{00(r-1)}\mathbf{m}_{(r-1)}^t \end{bmatrix} \mathbf{G}_{r-1}^t. \tag{34}$$

Taking into account the fact that

$$\mathbf{A}_{r-1} = \begin{bmatrix} b_{11r} & \bar{\mathbf{a}}_{1r}^t \\ \bar{\mathbf{a}}_{1r} & \mathbf{B}_r \end{bmatrix}, \tag{35}$$

it is easy to obtain

$$\mathbf{G}_r \begin{bmatrix} 1 & 0 & \mathbf{0} \\ 0 & 1 & \mathbf{0} \\ \mathbf{m}_{0r} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix} \begin{bmatrix} b_{00r} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_{r-1} + \mathbf{F}_{r-1} \end{bmatrix} \begin{bmatrix} 1 & 0 & \mathbf{m}_{0r}^t \\ 0 & 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{(r-2)x(r-2)} \end{bmatrix} \mathbf{G}_r^t = \mathbf{A}_r + \mathbf{F}_r + \mathbf{G}_r \begin{bmatrix} 0 & \mathbf{0} \\ \mathbf{0} & F_{r-1} \end{bmatrix} \mathbf{G}_r^t \tag{36}$$

and, thus,

$$\mathbf{F} = \mathbf{F}_n + \mathbf{G}_n \begin{bmatrix} 0 & \mathbf{0} \\ \mathbf{0} & \mathbf{F}_{n-1} \end{bmatrix} \mathbf{G}_n^t + \cdots + \mathbf{G}_n \mathbf{G}_{n-1} \ldots \mathbf{G}_{k+1} \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{F}_k \end{bmatrix} \mathbf{G}_{k+1}^t \ldots \mathbf{G}_{n-1}^t \mathbf{G}_n^t + \mathbf{G}_n \ldots \mathbf{G}_2 \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{F}_1 \end{bmatrix} \mathbf{G}_2^t \ldots \mathbf{G}_n^t. \tag{37}$$

For a single element in Eq. (37), we can write

$$\mathbf{G}_n \mathbf{G}_{n-1} \ldots \mathbf{G}_{k+1} \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{F}_k \end{bmatrix} \mathbf{G}_{k+1}^t \ldots \mathbf{G}_{n-1}^t \mathbf{G}_n^t = \mathbf{F}_{gk} \tag{38}$$

and

$$\mathbf{F}_{gk} = \begin{bmatrix} s_n^2 s_{n-1}^2 \ldots s_{k+1}^2 \epsilon_{00k} & \cdots & s_n \ldots s_{k+1} c_{k+1} \epsilon_{00k} & s_n \ldots s_{k+1} \epsilon_{01k} & s_n \ldots s_{k+1} (\boldsymbol{\epsilon}_{0(k+1)1}^t + \boldsymbol{\epsilon}_{0(k+1)2}^t) \\ s_n c_n s_{n-1} s_{n-2}^2 \ldots s_{k+1}^2 \epsilon_{00k} & \cdots & c_n s_{n-1} \ldots s_{k+1} c_{k+1} \epsilon_{00k} & c_n \ldots s_{k+1} \epsilon_{01k} & c_n \ldots s_{k+1} (\boldsymbol{\epsilon}_{0(k+1)1}^t + \boldsymbol{\epsilon}_{0(k+1)2}^t) \\ \ldots & \cdots & \cdots & \cdots & \cdots \\ s_n s_{n-1} \ldots s_{k+1} c_{k+1} \epsilon_{00k} & \cdots & c_r \epsilon_{00k} & c_r \epsilon_{01k} & c_r (\boldsymbol{\epsilon}_{0(k+1)1}^t + \boldsymbol{\epsilon}_{0(k+1)2}^t) \\ s_n s_{n-1} c_{n-1} s_{n-2}^2 \ldots s_{k+1}^2 \epsilon_{00k} & \cdots & s_n \ldots s_{k+1} c_{k+1} \epsilon_{00k} & s_n \ldots s_{k+1} \epsilon_{01k} & s_n \ldots s_{k+1} (\boldsymbol{\epsilon}_{0(k+1)1} + \boldsymbol{\epsilon}_{0(k+1)2}) \\ s_n s_{n-1} \ldots s_{k+1} \epsilon_{01k} & \cdots & c_r \epsilon_{01k} & \epsilon_{11k} & \boldsymbol{\epsilon}_{1r}^t \\ s_n \ldots s_{k+1} (\boldsymbol{\epsilon}_{0(k+1)1} + \boldsymbol{\epsilon}_{0(k+1)2}) & \cdots & c_{k+1} (\boldsymbol{\epsilon}_{0(k+1)1} + \boldsymbol{\epsilon}_{0(k+1)2}) & \boldsymbol{\epsilon}_{1r} & \boldsymbol{\epsilon}_{rs} + \boldsymbol{\epsilon}_{0r2} b_{00r} \mathbf{m}_{k+1}^t \end{bmatrix} \tag{39}$$

which can be bounded by

$$\begin{bmatrix} \left(\frac{\sqrt{2}}{2}\right)^{2(n-(k+1))} |\epsilon_{00k}| & \cdots & \left(\frac{\sqrt{2}}{2}\right)^{n-k-1} |\epsilon_{00k}| & \left(\frac{\sqrt{2}}{2}\right)^{n-k-1} |\epsilon_{01k}| & \left(\frac{\sqrt{2}}{2}\right)^{n-k-1} (|\boldsymbol{\epsilon}_{0(k+1)1}^t| + |\boldsymbol{\epsilon}_{0(k+1)2}^t|) \\ \left(\frac{\sqrt{2}}{2}\right)^{2(n-(k+1))-1} |\epsilon_{00k}| & \cdots & \left(\frac{\sqrt{2}}{2}\right)^{n-k-2} |\epsilon_{00k}| & \left(\frac{\sqrt{2}}{2}\right)^{n-k-2} |\epsilon_{01k}| & \left(\frac{\sqrt{2}}{2}\right)^{n-k-2} (|\boldsymbol{\epsilon}_{0(k+1)1}^t| + |\boldsymbol{\epsilon}_{0(k+1)2}^t|) \\ \ldots & \cdots & \cdots & \cdots & \cdots \\ \left(\frac{\sqrt{2}}{2}\right)^{n-k-1} |\epsilon_{00k}| & \cdots & |\epsilon_{00k}| & |\epsilon_{01k}| & |\boldsymbol{\epsilon}_{0(k+1)1}^t| + |\boldsymbol{\epsilon}_{0(k+1)2}^t| \\ \left(\frac{\sqrt{2}}{2}\right)^{n-k-2} |\epsilon_{01k}| & \cdots & |\epsilon_{01k}| & |\epsilon_{11k}| & |\boldsymbol{\epsilon}_{1r}^t| \\ \frac{(|\boldsymbol{\epsilon}_{0(k+1)1}| + |\boldsymbol{\epsilon}_{0(k+1)2}|)}{\sqrt{2}^{(n-k-2)}} & \cdots & |\boldsymbol{\epsilon}_{0(k+1)1}| + |\boldsymbol{\epsilon}_{0(k+1)2}| & |\boldsymbol{\epsilon}_{1r}| & |\boldsymbol{\epsilon}_{rs}| + |\boldsymbol{\epsilon}_{0r2} b_{00r} \mathbf{m}_{k+1}^t| \end{bmatrix}. \tag{40}$$

To obtain bounds on $|\epsilon_{00r}|$, $|\epsilon_{01r}|$, $|\epsilon_{11r}|$, $|\boldsymbol{\epsilon}_{0r1}|$, and $|\boldsymbol{\epsilon}_{1r}|$, we take into account the fact that these quantities appear in a Jacobi rotation. Taking account also of the fact that the rotation is performed in finite precision, so that the exact value of b_{01} will not be zero, it is easy to obtain these bounds:

$$|\epsilon_{00}| \le 3\mu_{\max} u + O(u^2), \tag{41}$$

$$|\epsilon_{11}| \le 3\mu_{\max} u + O(u^2), \tag{42}$$

$$|\epsilon_{01}| \le 4\mu_{\max} u + O(u^2), \tag{43}$$

$$|\epsilon_{0r1i}| \le 6.75\mu_{\max} u + O(u^2), \tag{44}$$

$$|\epsilon_{1ri}| \le 6.75\mu_{\max} u + O(u^2), \tag{45}$$

where $\mu_{\max}$ is the maximum absolute value of the elements obtained in the intermediate matrices during the factorization process. For the sake of simplicity, the more generous expression $6.75\mu_{\max} u$ will be used for all of these bounds. Considering the operations involved in the reduction, we obtain

$$|\epsilon_{0r2i}| \le \sqrt{2}\mu_{\max} u + O(u^2), \tag{46}$$

$$|\epsilon_{rsij} + \epsilon_{0r2i} b_{00r} m_{rj}| \le (2\sqrt{2} + 3)\mu_{\max} u + O(u^2). \tag{47}$$

From now on, the terms in u^2 will be disregarded. Taking this into account, we can write

$$|f_{rij}| \le u\mu_{\max} \left\{ \begin{array}{lll} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r-i+1)-(j-i)}, & i \le j < n-r, & r < n-j \le n-i \\ (6.75+\sqrt{2})\left(\frac{\sqrt{2}}{2}\right)^{n-r+1-i}, & i < n-r \le j, & n-j \le r < n-i \\ 6.75+\sqrt{2}, & j \ge i \le n-r \le i+1, & r \le n-i \le r+1 \\ \sqrt{2}+3, & n-r+1 < i \le j, & r > n-i+1 \ge n-j+1 \end{array} \right\}. \tag{48}$$

By performing a summation for each of the $\mathbf{F}_r$, we can write the following for each element of $\mathbf{F}$. For the terms in the main diagonal,

$$|f_{ii}| \le \left\{ \begin{array}{ll} u\mu_{\max}\left(\sum_{r=2}^{n-1} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r+1)} + 6.75+\sqrt{2}\right) \le u\mu_{\max}(9.75+\sqrt{2}), & i=0 \\ u\mu_{\max}\left(\sum_{r=2}^{n-i-1} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r-i+1)} + 6.75+\sqrt{2}+6.75+\sqrt{2}\right) \le u\mu_{\max}(16.5+2\sqrt{2}), & i=1 \\ u\mu_{\max}\left(16.5+2\sqrt{2}+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right) \le u\mu_{\max}(16.5+2\sqrt{2}+(2\sqrt{2}+3)(i-2)), & 2 \le i \le n-3 \\ u\mu_{\max}\left(2*6.75+2\sqrt{2}+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right) \le u\mu_{\max}(13.5+2\sqrt{2}+(2\sqrt{2}+3)(i-2)), & i=n-2 \\ u\mu_{\max}\left(6.75+\sqrt{2}+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right) \le u\mu_{\max}(6.75+\sqrt{2}+(2\sqrt{2}+3)(i-2)), & i=n-1 \end{array} \right\}. \tag{49}$$

For the terms satisfying $j = i+1$ (or $j = i-1$),

$$|f_{i(i+1)}| \le \left\{ \begin{array}{ll} u\mu_{\max}\left(\sum_{r=2}^{n-i-1} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r-i)+1} + 6.75+\sqrt{2}\right) = u\mu_{\max}\left(3\frac{\sqrt{2}}{2}+6.75+\sqrt{2}\right), & i=0 \\ u\mu_{\max}\left(\sum_{r=2}^{n-i-1} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r-i)+1} + 6.75+\sqrt{2}+6.75+\sqrt{2}\right) = u\mu_{\max}\left(7\frac{\sqrt{2}}{2}+13.5\right), & i=1 \\ u\mu_{\max}\left(7\frac{\sqrt{2}}{2}+13.5+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right) = u\mu_{\max}\left(7\frac{\sqrt{2}}{2}+13.5+(2\sqrt{2}+3)(i-2)\right), & 2 \le i \le n-3 \\ u\mu_{\max}\left(2*(6.75+\sqrt{2})+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right) = u\mu_{\max}\left(13.5+2\sqrt{2}+(2\sqrt{2}+3)(i-2)\right), & i=n-2 \\ u\mu_{\max}\left(6.75+\sqrt{2}+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right) = u\mu_{\max}\left(6.75+\sqrt{2}+(2\sqrt{2}+3)(i-2)\right), & i=n-1 \end{array} \right\}. \tag{50}$$

And, for all the others,

$$|f_{ij}| \le \left\{ \begin{array}{ll} u\mu_{\max}\left(\sum_{r=2}^{n-j-1} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r+1)-j} + \sum_{r=n-j}^{n-1}(6.76+\sqrt{2})\left(\frac{\sqrt{2}}{2}\right)^{n-r+1} + 6.75+\sqrt{2}\right), & i=0 \\ u\mu_{\max}\left(\sum_{r=2}^{n-j-1} 3\left(\frac{\sqrt{2}}{2}\right)^{2(n-r+1)-j} + \sum_{r=n-j}^{n-1}(6.76+\sqrt{2})\left(\frac{\sqrt{2}}{2}\right)^{n-r+1} + 2*(6.75+\sqrt{2})\right), & i=1 \\ u\mu_{\max}\left(24.25+11.25\sqrt{2}+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right), & 2 \le i \le n-3 \\ u\mu_{\max}\left(2*(6.76+\sqrt{2})+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right), & i=n-2 \\ u\mu_{\max}\left(6.76+\sqrt{2}+\sum_{r=n-i+2}^{n}(2\sqrt{2}+3)\right), & i=n-1 \end{array} \right\}, \tag{51}$$

$$|f_{ij}| \le \left\{ \begin{array}{cc} u\mu_{\max}\left(17.5 + 10.25\sqrt{2}\right), & i = 0 \\ u\mu_{\max}\left(24.25 + 11.25\sqrt{2}\right), & i = 1 \\ u\mu_{\max}\left(24.25 + 11.25\sqrt{2} + (i-2)(2\sqrt{2}+3)\right), & 2 \le i \le n-3 \\ u\mu_{\max}\left(13.5 + 2\sqrt{2} + (i-2)(2\sqrt{2}+3)\right), & i = n-2 \\ u\mu_{\max}\left(6.75 + \sqrt{2} + (i-2)(2\sqrt{2}+3)\right), & i = n-1 \end{array} \right\}. \tag{52}$$

If we take the sum of the elements of each column or row, we obtain

$$\sum_{i=0}^{n-1} |f_{ij}| \le \sum_{i=0}^{j-2} |f_{ij}| + |f_{(j-1)j}| + |f_{jj}| + |f_{j(j+1)}| + \sum_{r=j+2}^{n-1} |f_{jr}|. \tag{53}$$

For matrices above a size that can be determined, the largest sum occurs in the last column, for which

$$\begin{aligned} \sum_{i=0}^{n-1} |f_{i(n-1)}| &\le \sum_{i=0}^{n-3} |f_{i(n-1)}| + |f_{(n-2)(n-1)}| + |f_{(n-1)(n-1)}| \\ &\le (5.83n^2 + 11.02n - 52.56)u\mu_{\max}, \end{aligned} \tag{54}$$

which is a bound on $|\mathbf{F}|$.

5. Bound on the growth of elements

The steps presented by Wilkinson [11] will be followed here. We use the Hadamard inequation,

$$|\det \mathbf{A}| \le \left\{ \prod_{i=0}^{n-1} \sum_{i=0}^{n-1} a_{ij}^2 \right\}^{1/2} \le (\sqrt{n}\mu)^n, \tag{55}$$

where μ is the maximum absolute value of the elements in $\mathbf{A}$. For a given submatrix in step r, we can write

$$|\det \mathbf{A}_r| \le (\sqrt{r}\mu_r)^n. \tag{56}$$

Let $p_r = |b_{00r}|$ be the absolute value of the pivot obtained for the matrix $\mathbf{A}_r$. Taking into account the procedure employed to obtain p_r, we can write

$$p_r \ge \mu_r. \tag{57}$$

For the fully factorized matrix, we can state that

$$p_1 p_2 \ldots p_r = |\det \mathbf{A}_r| \le (\sqrt{r}\mu_r)^r \le (p_r\sqrt{r})^r. \tag{58}$$

Introducing

$$q_r = \log p_r, \tag{59}$$

we have

$$q_1 + q_2 + \cdots + q_{r-1} \le (r-1)q_r + \frac{r}{2}\log r. \tag{60}$$

By taking the sum of the equations for values $2 \le r \le n-1$, we can easily obtain

$$q_1 - \sum_{l=1}^{n-1} \frac{q_l}{n-1} \le \sum_{k=2}^{n-1} \frac{1}{2(k-1)} \log k. \tag{61}$$

And, taking into account

$$\log|\det A_r| = q_1 + q_2 + \cdots + q_n, \tag{62}$$

we can write

$$q_1 + \frac{q_n}{n-1} \le \frac{1}{n-1}\log|\det A| + \sum_{k=2}^{n-1} \frac{1}{2(k-1)} \log k. \tag{63}$$

Introducing

$$\log|\det A| \le n\log\sqrt{n} + n\log p_n = n\log\sqrt{n} + nq_n, \tag{64}$$

we obtain

$$q_1 - q_n = \frac{1}{2}\log n + \frac{1}{2(n-1)}\log n + \sum_{k=2}^{n-1} \frac{1}{2}\log k^{1/(k-1)}. \tag{65}$$

Introducing the known function

$$f(n) = \sqrt{\prod_{k=2}^{n} \frac{1}{k-1}}, \tag{66}$$

we can write

$$q_1 - q_n \le \frac{1}{2} \log n + \log(f(n)) \tag{67}$$

and, thus,

$$\frac{p_1}{p_n} \le \sqrt{n} f(n). \tag{68}$$

We can also write

$$\frac{p_k}{p_n} \le \sqrt{n-k+1} f(n-k+1) \le \sqrt{n} f(n), \tag{69}$$

which bounds the growth of the pivot after the rotation. We can state that $\mu_k \le p_k \le 2\mu_k$, and so we can write

$$\frac{\mu_1}{2\mu_n} \le \sqrt{n} f(n), \tag{70}$$

or

$$\mu_1 \le 2\mu_n \sqrt{n} f(n). \tag{71}$$

Taking into account the fact that μ_n is the largest absolute value in the elements of $\mathbf{A}$, we can state that $2\sqrt{n}f(n)$ is a bound on the growth of the elements in the process. As is known, the Bunch method using $\alpha = \alpha_0$ is bounded by $3.07(n-1)^{0.446}$ $\sqrt{n}f(n)$, and full-pivoted Gaussian elimination has a bound on its growth equal to $\sqrt{n}f(n)$.

6. Error analysis of the solution of a linear system of equations

Let

$$\mathbf{A}\mathbf{x} = \mathbf{y} \tag{72}$$

be a linear system of equations. The aim is to obtain a bound for a matrix $\mathbf{E}$ so that

$$(\mathbf{A} + \mathbf{E})\mathbf{x} = \mathbf{y}, \tag{73}$$

where $\mathbf{x}$ has been obtained in finite precision using the factorization algorithm presented here. The procedure can be decomposed into the following operations:

$$\mathbf{A} + \mathbf{F} = \mathbf{M}\mathbf{D}\mathbf{M}^t, \tag{74}$$

$$(\mathbf{M} + \delta\mathbf{M})\mathbf{u} = \mathbf{y}, \tag{75}$$

$$(\mathbf{D} + \delta\mathbf{D})\mathbf{v} = \mathbf{u}, \tag{76}$$

$$(\mathbf{M}^t + \Delta\mathbf{M}^t)\mathbf{x} = \mathbf{v}. \tag{77}$$

A bound on $\mathbf{F}$ has already been obtained. Bounds on the other error matrices will be presented later. Taking these equations into account, we can write

$$(\mathbf{M}\mathbf{D}\mathbf{M}^t - \mathbf{F} + \mathbf{E})\mathbf{x} = (\mathbf{M} + \delta\mathbf{M})(\mathbf{D} + \delta\mathbf{D})(\mathbf{M}^t + \Delta\mathbf{M}^t)\mathbf{x} \tag{78}$$

and

$$\mathbf{E} = \mathbf{F} + \mathbf{M}(\delta\mathbf{D}(\mathbf{M}^t + \Delta\mathbf{M}^t) + \mathbf{D}\,\Delta\mathbf{M}^t) + \delta\mathbf{M}((\mathbf{D} + \delta\mathbf{D})(\mathbf{M}^t + \Delta\mathbf{M}^t)). \tag{79}$$

Thus, we can bound $|\mathbf{E}|$:

$$|\mathbf{E}| \le |\mathbf{F}| + |\mathbf{M}|(|\delta\mathbf{D}|(|\mathbf{M}^t| + |\Delta\mathbf{M}^t|) + |\mathbf{D}|\,|\Delta\mathbf{M}^t|) + |\delta\mathbf{M}|((|\mathbf{D}| + |\delta\mathbf{D}|)(|\mathbf{M}^t| + |\Delta\mathbf{M}^t|)). \tag{80}$$

7. Bounds on $|\mathbf{D}|$ and $|\delta\mathbf{D}|$

As presented earlier,

$$p_r = |b_{00r}| \le 2\mu_r, \tag{81}$$

where μ_r is the maximum absolute value of the elements of $\mathbf{A}_r$. The kth element of $\mathbf{D}$ is obtained from $d_k v_k = u_k$. We can write

$$(d_k + \delta d_k) v_k = u_k, \tag{82}$$

with

$$|d_k| \leq 2\mu_r \tag{83}$$

and

$$|\delta d_k| \leq 2\mu_r u. \tag{84}$$

So, we can write

$$|\mathbf{D}| \leq 2\mu \tag{85}$$

and

$$|\delta \mathbf{D}| \leq 2\mu u. \tag{86}$$

8. Bounds on |M|, |δM|, and |△M|

As the permutations in the matrices have already been considered in the analysis of the factorization, we can omit these permutations without losing generality. The full operation has the form

$$\mathbf{Mu} = \begin{bmatrix} c_1 & s_1c_2 & s_1s_2c_3 & \cdots & c_{n-1}s_{n-2}\ldots s_1 & s_{n-1}\ldots s_1 \\ -s_1 & c_1c_2 & c_1s_2c_3 & \cdots & c_{n-2}s_{n-2}\ldots s_2c_1 & s_{n-1}\ldots s_2c_1 \\ l_20 & -s_2 & c_2c_3 & \cdots & c_{n-1}s_{n-2}\ldots s_3c_2 & s_{n-1}\ldots s_3c_2 \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & c_{n-1}c_{n-2} & s_{n-1}c_{n-2} \\ l_{(n-1)0} & l_{(n-1)1} & l_{(n-1)2} & \cdots & -s_{n-1} & c_{n-1} \end{bmatrix} \begin{Bmatrix} u_0 \\ u_1 \\ \cdots \\ u_{n-1} \end{Bmatrix}$$

$$= \begin{Bmatrix} y_0 \\ y_1 \\ \cdots \\ y_{n-1} \end{Bmatrix}. \tag{87}$$

In the ith iteration of the process,

$$\mathbf{G}_i \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & c_i & s_ic_{i+1} & s_is_{i+1}c_{i+2} & \cdots & s_{n-1}\ldots s_{i+1} \\ \mathbf{0} & -s_i & c_ic_{i+1} & c_is_{i+1}c_{i+2} & \cdots & s_{n-1}\ldots s_{i+1}s_i \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \mathbf{0} & l_{(n-1)i} & l_{(n-1)(i+1)} & l_{(n-1)(i+2)} & \cdots & c_{n-1} \end{bmatrix} \begin{Bmatrix} u_0 \\ \cdots \\ u_i \\ u_{i+1} \\ \cdots \\ u_{n-1} \end{Bmatrix} = \mathbf{G}_i \left(\begin{Bmatrix} u_0 \\ u_1 \\ \cdots \\ y_i^{(i)} \\ y_{i+1}^{(i)} \\ \cdots \\ y_{n-1}^{(i)} \end{Bmatrix} - u_{i-1} \begin{Bmatrix} 0 \\ 0 \\ \cdots \\ 0 \\ l_{(i+1)(i-1)} \\ \cdots \\ l_{(n-1)(i-1)} \end{Bmatrix} \right), \tag{88}$$

where

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{I}_{(n-r)x(n-r)} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & c_i & -s_i & \mathbf{0} \\ \mathbf{0} & s_i & c_i & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \end{bmatrix}. \tag{89}$$

One use this equation to solve the problem iteratively, but if a floating-point version with an accumulator is to be developed, it is possible to use the equivalent expressions

$$y_k^a = y_k^r - \sum_{j=0}^{k-2} u_j l_{kj}, \tag{90}$$

$$u_{k-1} = c_k y_{k-1}^r - s_k y_k^a, \tag{91}$$

$$y_k^r = s_k y_{k-1}^r + c_k y_k^a. \tag{92}$$

To perform an error analysis, we introduce $i_k = 1$:

$$y_k^a = \frac{1}{i_k}\left(y_k^r - \sum_{j=0}^{k-2} u_j l_{kj} \right), \tag{93}$$

$$u_{k-1} = c_k y_{k-1}^r - s_k y_k^a, \tag{94}$$

$$y_k^r = s_k y_{k-1}^r + c_k y_k^a, \tag{95}$$

which is equivalent to writing Eq. (88) as

$$\mathbf{G}_i \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & c_i & s_i c_{i+1} & s_i s_{i+1} c_{i+2} & \cdots & s_{n-1}..s_{i+1} \\ \mathbf{0} & -s_i i_{k+1} & c_i i_{k+1} c_{i+1} & c_i i_{k+1} s_{i+1} c_{i+2} & \cdots & s_{n-1} \ldots s_{i+1} s_i i_{k+1} \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \mathbf{0} & l_{(n-1)i} & l_{(n-1)(i+1)} & l_{(n-1)(i+2)} & \cdots & c_{n-1} i_k \end{bmatrix} \begin{Bmatrix} u_0 \\ \cdots \\ u_i \\ u_{i+1} \\ .. \\ u_{n-1} \end{Bmatrix}$$

$$= \mathbf{G}_i \left(\begin{Bmatrix} u_0 \\ u_1 \\ \cdots \\ y_i^{(i)} \\ y_{i+1}^{(i)} \\ \cdots \\ y_{n-1}^{(i)} \end{Bmatrix} - u_{i-1} \begin{Bmatrix} 0 \\ 0 \\ \cdots \\ 0 \\ l_{(i+1)(i-1)} \\ \cdots \\ l_{(n-1)(i-1)} \end{Bmatrix} \right). \tag{96}$$

So, in finite precision (see [11]),

$$y_k^a = \frac{1}{i_k(1+\xi)(1+\epsilon)} \left(y_k^r - \sum_{j=0}^{k-2} u_j l_k j \frac{(1+\xi_{kj})}{(1+\epsilon)} \right)$$

$$= (i_k + \delta i_k) \left(y_k^r - \sum_{j=0}^{k-2} u_j (l_{kj} + \delta l_{kj}) \right), \tag{97}$$

where

$$|\xi| \le u, \tag{98}$$

$$|\epsilon| \le u, \tag{99}$$

$$|\xi_k j| \le (k+1-j)u, \tag{100}$$

and

$$|\delta i_k| \le 2u, \tag{101}$$

$$|\delta l_{kj}| \le (k+2-j)u. \tag{102}$$

We can write

$$\delta \mathbf{M} = \begin{bmatrix} \delta c_1 & \delta s_1 c_2 + \delta c_2 s_1 & \delta s_2 s_1 c_3 + \delta s_1 s_2 c_3 + \delta c_3 s_1 s_2 & \cdots & \delta s_{n-1} s_{n-2} \ldots s_1 + \delta s_1 s_{n-1} \ldots s_2 \\ -\delta s_1 & \delta c_1 c_2 + \delta c_2 c_1 & \delta c_1 s_2 c_3 + \delta s_2 c_1 c_3 + \delta c_3 c_1 s_3 & \cdots & \delta s_{n-1} s_{n-2} \ldots s_2 c_1 + \cdots + \delta c_1 s_{n-1} \ldots s_2 \\ \delta l_{20} & -\delta s_2 - \delta i_3 s_2 & \delta c_2 c_3 + \delta c_3 c_2 + \delta i_3 c_2 c_3 & \cdots & \delta s_{n-1} s_{n-2} \ldots s_3 c_2 + \cdots + \delta c_2 s_{n-1} \ldots s_3 + \delta i_2 s_{n-1} \ldots s_3 c_2 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ \delta l_{(n-1)0} & \delta l_{(n-1)1} & \delta l_{(n-1)2} & \cdots & \delta c_{n-1} + \delta i_{n-1} \end{bmatrix}. \tag{103}$$

We just need to find bounds on δc_i and δs_i. We must take into account the fact that, in order to increase the stability, it is better to write Eqs. (94) and (95) as

$$u_{k-1} = y_{k-1} - s_k (y_k^a + \rho y_{k-1}^r), \tag{104}$$

$$y_k^r = y_k^a + s(y_{k-1}^r - \rho y_k^a). \tag{105}$$

It is easy to find that, with these conditions,

$$|\delta s_k| \le 13.5u, \tag{106}$$

$$|\delta c_k| \le 13.5u, \tag{107}$$

so that, taking account of the fact that $l_{ij} \le \sqrt{2}$,

$$|\delta M| \le u \begin{bmatrix} 13.5 & 13.5\left(1+\frac{\sqrt{2}}{2}\right) & 13.5\left(\left(2\frac{\sqrt{2}}{2}\right)\right) + \left(\frac{\sqrt{2}}{2}\right)^2 & \cdots & 13.5(n-1)\left(\frac{\sqrt{2}}{2}\right)^{(n-2)} \\ 13.5 & 13.5 \cdot 2 & 13.5\left(2\left(\frac{\sqrt{2}}{2}\right)+1\right) & \cdots & 13.5\left((n-2)\left(\frac{\sqrt{2}}{2}\right)^{n-3} + \left(\frac{\sqrt{2}}{2}\right)^{n-2}\right) \\ 4\sqrt{2} & 13.5 + \frac{\sqrt{2}}{2} & 13.5 \cdot 2 + 1 & \cdots & 13.5\left((n-3)\left(\frac{\sqrt{2}}{2}\right)^{n-4} + \left(\frac{\sqrt{2}}{2}\right)^{n-3}\right) + \left(\frac{\sqrt{2}}{2}\right)^{(n-3)} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ (n+1)\sqrt{2} & n\sqrt{2} & (n-1)\sqrt{2} & \cdots & 13.5+1 \end{bmatrix}. \tag{108}$$

Table 1
Stages and operation count for each individual stage for a submatrix of size k.

Stage	Memory accesses	Products	Divisions	Additions and subtractions	Comparisons	Square roots
Pivot search	$k^2/2+k/2$	0	0	0	$k^2/2+k/2-1$	0
Row/column	$4k$	0	0	0	0	0
Interchanges	$4k$	0	0	0	0	0
Rotation	$2k-1$	$4k-6$	2	$4k-3$	0	1
Reduction	$k^2/2-k/2+1$	$k^2/2-k/2$	1	$k^2/2-3k/2+1$	0	0
Total	k^2+6k	$k^2/2+(7/2)k-6$	3	$k^2/2+(5/2)k-2$	$k^2/2+k/2-1$	1

We can write

$$|\delta m_{ij}| \le u \left\{ \begin{array}{cc} 13.5(j-i)\left(\frac{\sqrt{2}}{2}\right)^{j-i-1} + 14.5\left(\frac{\sqrt{2}}{2}\right)^{j-i}, & j > i+1 \\ 13.5 + 14.5\left(\frac{\sqrt{2}}{2}\right), & j = i+1 \\ 28, & j = i \\ 14.5, & i = j+1 \\ (i+2-j)\sqrt{2}, & i > j \end{array} \right\}. \tag{109}$$

If we denote the jth column of $\delta\mathbf{M}$ by $\delta\mathbf{m}_j$, then

$$|\delta\mathbf{m}_j| \le u\left(\sum_{i=0}^{j-2}\left(13.5(j-i)\left(\frac{\sqrt{2}}{2}\right)^{j-i-1} + 14.5\left(\frac{\sqrt{2}}{2}\right)^{j-i}\right) + \left(13.5 + 14.5\frac{\sqrt{2}}{2}\right) + 28 + 14.5 + \sum_{i=j+2}^{n-1}(i+2-j)\sqrt{2}\right). \tag{110}$$

For values of n above some value that can be determined, the largest value occurs for $j = 0$, and thus

$$|\delta\mathbf{M}| = |\delta\mathbf{m}_j| \le u\left(28 + 14.5 + \sum_{i=2}^{n-1}(i+2)\sqrt{2}\right) = u(0.707n^2 + 2.1213n + 39). \tag{111}$$

Using the same model, the Bunch method would give

$$|\delta\mathbf{M}| \le u(1.3889n^2 + 6.95n - 7.334). \tag{112}$$

The full solution operation has an error bound that can be obtained from

$$|\mathbf{E}| = |\mathbf{F}| + |\delta\mathbf{M}|\,|\mathbf{D}\,\delta\mathbf{D}|\,|\mathbf{M}^t + \Delta\mathbf{M}^t| + |\mathbf{M} + \delta\mathbf{M}|\,|\delta\mathbf{D}|\,|\mathbf{M}^t + \Delta\mathbf{M}^t| + |\mathbf{M} + \delta\mathbf{M}^t|\,|\mathbf{D} + \delta\mathbf{D}|\,|\Delta\mathbf{M}^t|. \tag{113}$$

Although $\Delta\mathbf{M}^t$ and $\delta\mathbf{M}$ are not equal, their norms have the same bound. We have already obtained the following results:

$$|\mathbf{F}| \le (5.83n^2 + 11.02n - 53.56)\mu_{\max}u, \tag{114}$$

$$|\delta\mathbf{M}| \le (0.707n^2 + 2.1213n + 39)u \ge |\Delta\mathbf{M}|, \tag{115}$$

$$|\mathbf{M}| \le 1 + (n-2)1.41421, \tag{116}$$

$$|\delta\mathbf{D}| \le 2\mu_{\max}u, \tag{117}$$

$$|\mathbf{D}| \le 2\mu_{\max}. \tag{118}$$

So, we obtain

$$|\mathbf{E}| \le u\mu_{\max}(4n^3 + 12.17n^2 + 2058n - 332.048). \tag{119}$$

9. Operation count

First, the count for a submatrix of size k will be addressed, and then the total for each step will be computed. The stages and the operation counts for each stage are listed in Table 1.

Obviously, some numbers depend on the implementation selected. The total number of operations is obtained by taking the sum of the numbers of operations from 2 to n, which delivers the numbers in Table 2. It can be seen that the dominant terms are equal to those obtained for the Bunch–Parlett method, but there is a difference in the n^2 terms.

Table 2
Total operation counts.

Operation	Count
Memory accesses	$n^3/3 + (7/2)n^2 + (19/6)n - 7$
Products	$n^3/6 + 2n^2 - (25/6)n + 2$
Divisions	$3n - 3$
Additions/subtractions	$n^3/6 + (3/2)n^2 - (2/3)n - 1$
Comparisons	$n^3/6 + (1/2)n^2 - (2/3)n$
Square roots	$n - 1$

Table 3
Decomposition of random matrix: reconstruction error.

Size	Bunch–Kaufman	Bunch–Parlett	Rotated
10	1.52503e−15	1.22985e−15	1.0703e−15
100	8.01932e−14	4.57293e−14	3.2322e−14
1 000	4.96186e−12	2.46425e−12	1.78e−12
10 000	3.25498e−10	1.61790e−10	1.21621e−10

Table 4
Decomposition of random matrix: solution for a random vector.

Size	Bunch–Kaufman	Bunch–Parlett	Rotated
10	5.98312e−15	5.01387e−15	4.43759e−15
100	9.29895e−14	6.75799e−14	4.93276e−14
1 000	6.86688e−12	2.22289e−12	1.64536e−12
10 000	3.10212e−11	1.64892e−11	1.31949e−11

Table 5
Decomposition of random matrix: approximate solution time(s).

Size	Bunch–Kaufman	Bunch–Parlett	Rotated
10	3.80427e−06	3.30136e−06	4.77665e−06
100	0.000231683	0.000511746	0.000537695
1 000	0.166351	0.377934	0.38176
10 000	212.27945	442.60155	442.98795

10. Numerical experiments

As an improved bound for the error growth is obtained in relation to that of the Bunch–Parlett method, it is clear that, in a worst case scenario, the presented method should outperform the former in terms of accuracy, but one also wants to find if in a general scenario an improvement is obtained. As can be seen from the references (see, for example, [6]), a globally accepted matrix test suite has yet to be developed.

The experiment was performed with random matrices generated using a random number generator, with values in $\Re[-1, 1]$. Additionally, a random vector with elements generated in the same way was produced for each matrix; this was used as the solution for a system generated by multiplying this vector by the original matrix. In order to compute the errors in the most correct way possible, error calculations were done in quadruple precision. Afterwards, we tried to find the solution of the system generated, which was also used to measure the error. The results were averaged over 20 000 tests (with the exception of the $10\,000 \times 10\,000$ sized matrices, where only 200 tests have been performed, due to the high computational cost required). The size of the matrices appears in the first column. The compared methods are Bunch–Kaufman, as implemented in Lapack, Bunch–Parlett and the rotated pivoting scheme presented in this paper. Other methods such as Aasen tridiagonalization or Parlett–Reid have not been included due to two reasons: The results are not comparable, as they only lead to tridiagonalization, and not to a complete diagonalization, which delivers a result not directly usable for problems such as inertia calculations and more, and due to the fact that they have extensively been tested against Bunch–Parlett, with similar performance. The hardware employed included an Intel(R) Xeon(R) E5645 CPU running at 2.40 GHz, and the programming was single-threaded. For the random number generator, we used the MT19937 generator of Matsumoto and Nishimura implemented in the GNU Scientific Library [12], with the second revision of the seeding procedure introduced in 2002 by the same authors in their code.

The results are presented in Tables 3–5.

The reconstruction error shows a clear tendency, favoring the rotated factorization, but in a very slight factor. The solution of the linear system shows similar behavior, but with larger dispersion. The solution time is quite similar, slightly favoring the Bunch–Parlett method. This is mainly due to the fact that the use of 2×2 blocks in the Bunch–Parlett method translates into a marked reduction in the factorization time, due to the elimination of a pivot search. This effect vanishes as the size of the matrices increases.

11. Conclusions and future work

A new, alternative pivoting technique has been presented for the solution of symmetrical systems of equations. It provides a bound on the growth of the elements equal to $2\sqrt{n}f(n)$, which represents an improvement over the Bunch–Parlett algorithm. This improvement comes at the cost of a marginal increment in the total operation costs, and an increased size of the pivot index vector. Numerical experiments have shown a slight improvement in the error obtained in the factorization of random matrices and the solution of linear systems of equations. These conclusions suggest future work on developing a partial-pivoting scheme for the algorithm, which should deliver an improvement in accuracy compared with the Bunch–Kaufman algorithm with a reasonable increase in the computational cost.

Acknowledgments

The authors wish to acknowledge the financial support received from the Department of Research and Universities of the Basque Government (IT-432-10) and the Spanish Ministry of Science and Innovation, through the research project reference DPI2009-07900, "Methods for the Analysis and Design of Variable Geometry Trusses in Morphing Aircraft Applications". The work was also supported by UPV/EHU under program UFI 11/29.

References

[1] C. Ashcraft, R.G. Grimes, J.G. Lewis, Accurate symmetric indefinite linear equation solvers, SIAM J. Matrix Anal. Appl. 20 (1998) 513–561.
[2] N.J. Higham, Stability of the diagonal pivoting method with partial pivoting, SIAM J. Matrix Anal. Appl. 18 (1) (1997) 52–65.
[3] A. Druinsky, S. Toledo, The growth-factor bound for the Bunch–Kaufmann factorization is tight, SIAM J. Matrix Anal. Appl. 32 (3) (2011) 928–937.
[4] J.W. Demmel, N.J. Higham, Stability of block algorithms with fast level-3 BLAS, ACM Trans. Math. Software 18 (1992) 274–292.
[5] J.W. Demmel, N.J. Higham, R.S. Schreiber, Stability of block LU factorization, Numer. Linear Algebra Appl. 2 (1995) 173–190.
[6] N.J. Higham, Accuracy and Stability of Numerical Algorithms, SIAM, Philadelphia, 1996.
[7] J.R. Bunch, L. Kaufman, Some stable methods for calculating inertia and solving symmetric linear systems, Math. Comp. 31 (1977) 163–179.
[8] J.R. Bunch, B.N. Parlett, Direct methods for solving symmetric indefinite systems of linear equations, SIAM J. Numer. Anal. 8 (1971) 639–655.
[9] S. Hammarling, A survey of numerical aspects of plane rotations, MIMS EPrint 2008.69, Manchester Institute for Mathematical Sciences, University of Manchester, 1977.
[10] G.H. Golub, C.F. van Loan, Matrix Computations, third ed., Johns Hopkins University Press, ISBN: 978-0-8018-5414-9, 1996.
[11] J.H. Wilkinson, Rounding Errors in Algebraic Processes, Prentice-Hall, Englewood Cliffs, NJ, 1963.
[12] M. Matsumoto, T. Nishimura, Mersenne Twister: a 623-dimensionally equidistributed uniform pseudorandom number generator, ACM Trans. Model. Comput. Simul. 8 (1) (1998) 3–30.